ADS205 Tutorial 4 - Context-Free Grammars

October 7, 2026

1 ADS205 — Decision, Computation and Language

2 Tutorial 4 — Context-Free Grammars

Solutions are folded beneath each problem — click the arrow to expand them. Attempt the problem first.

2.1 Reference card

A **context-free grammar** is a tuple

$$G = (V, \Sigma, R, S),$$

where V is a finite set of variables, Σ is a finite set of terminals, R is a finite set of rules $A \to \alpha$, and $S \in V$ is the start variable.

Notation	Meaning
$uAv \Rightarrow u\alpha v$	one derivation step using $A \to \alpha$
$\Rightarrow^*$	zero or more derivation steps
$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$	the language generated by G
Leftmost derivation	always replace the leftmost remaining variable
Rightmost derivation	always replace the rightmost remaining variable
Ambiguous grammar	some string has two distinct parse trees, equivalently two distinct leftmost derivations

A grammar may be ambiguous even when another unambiguous grammar generates the same language.

---

3 Part A — Reading and constructing CFGs

3.1 Problem 1 — Concepts and counterexamples

Answer each question and justify your answer.

1. Two grammars have the same variables, terminals, and rules but different start variables. Must they generate the same language?
2. If L is context-free and $L' \subseteq L$, must L' be context-free?
3. Can an unambiguous grammar have a leftmost derivation and a different rightmost derivation for the same string?
4. Does every sequence of terminals and variables that can appear during a derivation belong to $L(G)$?

Solution — click to expand

1. **No.** The start variable determines where derivations begin. For rules $S \to 0A$ and $A \to 1$, starting at S generates `01`, while starting at A generates `1`.
2. **No.** Context-free languages are not closed under taking arbitrary subsets. For example, $\{0,1\}^*$ is context-free, but its subset $\{0^n1^n0^n \mid n \geq 0\}$ is not context-free.
3. **Yes.** They may apply the same production choices in a different order and still correspond to one parse tree.
4. **No.** A sentential form may still contain variables. Only terminal strings derived from S belong to $L(G)$.

3.2 Problem 2 — Language and derivation

Consider the grammar

$$S \to aSb \mid \varepsilon.$$

1. Give a derivation of `aaabbb`.
2. Describe $L(G)$ precisely.
3. Explain why `aabbb`, `abab`, and `ba` are not generated.
4. Is the grammar ambiguous?

Solution — click to expand

1. $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$.
2. $L(G) = \{a^nb^n \mid n \geq 0\}$.
3. Every use of $S \to aSb$ adds exactly one `a` to the left and one `b` to the right. Therefore the counts must match, and every `a` must precede every `b`.
4. The grammar is unambiguous. The length of the target string determines how many times $S \to aSb$ must be used, followed by one use of $S \to \varepsilon$.

3.3 Problem 3 — A CFG for strings ending in `a`

Give a CFG for

$$L = \{w \in \{a,b\}^* \mid w \text{ ends with } a\}.$$

Show derivations of `a`, `ba`, and `abba`. Explain why ε and `abb` cannot be generated.

Solution — click to expand

One suitable grammar is

$$S \to aS \mid bS \mid a.$$

Example derivations:

$$S \Rightarrow a,$$

$$S \Rightarrow bS \Rightarrow ba,$$

$$S \Rightarrow aS \Rightarrow abS \Rightarrow abbS \Rightarrow abba.$$

Every derivation finishes with $S \to a$, so every generated string ends in `a`. The grammar cannot generate ε. It cannot finish a derivation with `b`, so it cannot generate `abb`.

3.4 Problem 4 — Matching one symbol with two symbols

Construct a CFG for

$$L = \{0^n 1^{2n} \mid n \geq 0\}.$$

Show a derivation of `001111` and explain why the order of the terminals is correct.

Solution — click to expand

Use

$$S \to 0S11 \mid \varepsilon.$$

Then

$$S \Rightarrow 0S11 \Rightarrow 00S1111 \Rightarrow 001111.$$

Each recursive step adds one `0` at the far left and two `1`s at the far right. After n recursive steps, replacing S by ε gives exactly $0^n 1^{2n}$.

---

4 Part B — Derivations, parse trees, and ambiguity

4.1 Problem 5 — A parse tree and two derivation orders

Let

$$G: \quad S \to 00S1 \mid \varepsilon.$$

1. Draw a parse tree for `000011`.

2. Write its leftmost derivation.
3. Describe $L(G)$.

Solution — click to expand

The derivation is

$$S \Rightarrow 00S1 \Rightarrow 0000S11 \Rightarrow 000011.$$

There is only one variable in every sentential form, so the leftmost and rightmost derivations are identical. A text version of the parse tree is

```
S
  0
  0
  S
      0
      0
      S

      1
  1
```

Each recursive step adds two `0`s and one `1`, so

$$L(G) = \{0^{2n}1^n \mid n \geq 0\}.$$

4.2 Problem 6 — Detect and remove ambiguity

Consider

$$G_1: \quad S \to S + S \mid a,$$

where `+` is a terminal symbol.

1. Give two distinct leftmost derivations of `a+a+a`.
2. What are the two corresponding groupings?
3. Give an unambiguous grammar for the same language.

Solution — click to expand

Two leftmost derivations are

$$S \Rightarrow S + S \Rightarrow S + S + S \Rightarrow a + S + S \Rightarrow a + a + S \Rightarrow a + a + a,$$

and

$$S \Rightarrow S + S \Rightarrow a + S \Rightarrow a + S + S \Rightarrow a + a + S \Rightarrow a + a + a.$$

They correspond to $(a+a)+a$ and $a+(a+a)$. An unambiguous grammar for the same language is

$$S \to S + a \mid a,$$

which fixes left associativity. We could instead use $S \to a + S \mid a$ to fix right associativity.

To prove that the two grammars describe the same language, we could have two manners:

- Show that the two grammars correspond to the same set notation.

  For $G_1 : S \to S + S \mid a$, whenever we use the rule $S \to S + S$, we generate a `+` symbol and a new `S`, i.e., the number of the rule $S \to S + S$ being used equals the number of `+` and the number of `S` newly added.

  assume that we use the rule $S \to S + S$ n times during generation, the resultant string has n `+` and $n + 1$ `S`, i.e., $S \Rightarrow S(+S)^n$.

  Then we use the rule $S \to a$ $n+1$ times to get the string only with terminals, i.e., $S \Rightarrow a(+a)^n$. Then we have the set notation description of the language described by G_1.

  For $G_2 : S \to S + a \mid a$, similarly, when we use the rule $S \to S + a$, we generate a `+` symbol and a new `a`, i.e., the number of the rule $S \to S + a$ being used equals the number of `+` and the number of `a` newly added.

  assume that we use the rule $S \to S + a$ n times during generation, the resultant string has n `+` and n `a`, i.e., $S \Rightarrow S(+a)^n$.

  Then we use the rule $S \to a$ once, and we can get $S \Rightarrow a(+a)^n$.

  Therefore we prove that $L(G_1) = L(G_2)$.

- Show that the two grammars' corresponding languages are subsets of each other.
  - First show that $L(G_2) \subseteq L(G_1)$.

    The rule $S \to S + a$ in G_2 can be implemented by usage of two rules in G_1, i.e., using $S \to S + S$ once and $S \to a$ once.

    Since G_2 only has the rule $S \to S + a$ to recursively introduce new variables and the other rule also appears in G_1, all the strings generated by G_2 can be generated by G_1 by using the two rules in G_1 more times.

    Hence, $L(G_2) \subseteq L(G_1)$.
  - Then the other direction $L(G_1) \subseteq L(G_2)$.

    For $G_1 : S \to S + S \mid a$, we assume that we use the rule $S \to S + S$ n times when generating a string, then the string looks like $S \to S + S + \cdots + S$, where we have in total n `+` and $n + 1$ `S`.

    Then we use the rule $S \to a$ to eliminate the variable `S`, and we have $S \to a + a + \cdots + a$ with n `+` and $n + 1$ `a`.

    Considering $G_2 : S \to S + a \mid a$,, to generate the above string generated by G_1, we could use the rule $S \to S + a$ n times to get $S \to S + a + a + \cdots + a$ with a `S`, n `+` and n `a`.

And then we use the rule $S \to a$ once to get the target string $S \to a + a + a + \cdots + a$ with n `+` and $n + 1$ `a`.

Therefore, all the strings generated by G_1 can be generated by G_2, i.e., $L(G_1) \subseteq L(G_2)$.

– Given the above, we conclude that $L(G_1) = L(G_2)$.

**

4.3 Problem 7 — Add exponentiation to an expression grammar

Start from the usual grammar where `*` has higher precedence than `+` and both associate to the left. Add an exponentiation operator `↑` such that

- `↑` has higher precedence than `*`;
- `↑` associates to the right. (e.g., for `a↑b↑c`, we need to compute `b↑c` first, and then `a↑(b↑c)`)

Use your grammar to explain the structures of `a*b↑c` and `a↑b↑c`.

Reference CFG with `+` and `*` where `*` has higher precedence:

$$E \to E + T \mid T,$$
$$T \to T * F \mid F,$$
$$F \to (E) \mid a \mid b.$$

Solution — click to expand

One suitable grammar is

$$E \to E + T \mid T,$$
$$T \to T * P \mid P,$$
$$P \to F \uparrow P \mid F,$$
$$F \to (E) \mid a \mid b \mid c.$$

Seperate the three computation operators into three levels according to their precedence, i.e., `+` is the top level corresponding to the variable `E`, `*` is the second level corresponding to the variable `T`, and `↑` is the bottom level corresponding to the variable `P`. In this way, `↑` is computed first, and then `*`, and finally `+`. Beyond this, we need another level for the computation unit, which could either be something surrounded by parentheses or the terminals, i.e., so we have the variable `F`.

The separate P level gives exponentiation higher precedence than multiplication, so `a*b↑c` groups as $a * (b \uparrow c)$. The right-recursive rule $P \to F \uparrow P$ makes `a↑b↑c` group as $a \uparrow (b \uparrow c)$.

The difference between associating to the left and to the right is **putting the variable that can continue breaking up the same operator to the left or right of the operator**.

For example, $E \to E + T$, we assume that `+` associates to the left and the variable `E` can be broken up to introduce another `+`, so we put `E` to the left of `+`.

For $P \to F \uparrow P$, the operator $\uparrow$ associates to the right, and the variable `P` can be broken up to introduce another $\uparrow$, so we put the variable `P` to the right of $\uparrow$.

In other words, in a certain level of a parse tree, the direction of the association let the machine knows which should be computed first as a whole unit before conducting original operator in this level. The variable that can be broken up to grow deeper should be computed first.

4.4 Optional Tock exploration

The course author's grammar API constructs CFGs from readable production lines. `any_parse` returns one parse tree, while `all_parses` lets us expose ambiguity by enumerating distinct parses of the same word.

[ ]:
```
# Install once in a fresh environment, then import Tock.
# %pip install -q tock
from tock import *

ends_in_a = Grammar.from_lines([
    "S -> a S",
    "S -> b S",
    "S -> a",
])
zero_twice_one = Grammar.from_lines([
    "S -> 0 S 1 1",
    "S -> &",
])

for grammar, samples in [
    (ends_in_a, ["a", "b a", "a b a"]),
    (zero_twice_one, ["&", "0 1 1", "0 0 1 1 1 1"]),
]:
    display(grammar)
    for word in samples:
        display(any_parse(grammar, word))
```

[ ]:
```
# Tock identifiers use `plus` here to represent the terminal +.
ambiguous_plus = Grammar.from_lines([
    "S -> S plus S",
    "S -> a",
])

word = "a plus a plus a"
parses = list(all_parses(ambiguous_plus, word))
print(f"{word!r} has {len(parses)} parse trees")
for tree in parses:
    display(tree)

assert len(parses) == 2
```

4.5 What to take away

- The start variable is part of a CFG and can change the generated language.
- Recursive rules express dependencies such as matching counts and nesting.
- A parse tree records structural choices. Leftmost and rightmost derivations may list those choices in different orders.
- To prove ambiguity, find one string with two parse trees or two leftmost derivations.
- Ambiguity belongs to a grammar. Inherent ambiguity belongs to a language.
- Expression grammars use separate variables to encode precedence and associativity.